# Predicting Latency and Freshness of Cached Client- and Server-Rendered Web Applications

Which Cache Layer Answers, What It Costs, and Which Policy to Choose

ISTIAK AHMED TASHRIF, Sonargaon University, Bangladesh

Whether a web page shows its data sooner with client-side rendering (CSR) or server-side rendering (SSR), and how often that data is out of date, depends on which of several stacked caches answers each request, on the network, and on how much work the server does. We present a predictor that, before a workload runs, derives for every user action which cache layer serves it (browser, framework, application cache or database), the time until its data is visible, and the probability that it shows stale data. Its inputs are the cache policies, a description of the workload (its session distribution and write process, as could be estimated from past traffic), a one-time calibration without writes (16–36 runs per application and host) and round-trip times probed before each run. Every prediction is written before its run, on conditions never used for fitting; every version is tested again on an independent replication, and all versions, including failures, are reported. After calibration in each environment, latency is predicted within 0.2–1.8% mean error across emulated paths, one real relayed internet path in three configurations, a second framework, a heavier page, and an independently written second application (RealWorld Conduit) on two hosts. Pooled stale-view shares are predicted within 0.006–0.026 absolute error; stricter per-cell and calibration criteria were first missed, and a corrected model passed them on its two fresh tests. Used to choose among 27 rendering and caching policies under a staleness limit, the predictions selected policies within 2% of the observed best in all 32 decisions without a limit violation, matching live tuning with one pilot run per candidate where both were run, whereas framework-default TTLs violated the limit in 29 of 32. A retrospective comparison locates this decision quality: given the same predicted serving layers, a ridge regression chooses as the model does, but without the model's session timeline, calibration-frequency serving layers raise the worst regret to 13.7% and a single-layer staleness rule violates the limit in 12 of 32 decisions. With limits placed at the predicted boundary, a selector with a staleness margin made no violation in 16 decisions, but its mean regret in the replication (5.8%) exceeded the pre-registered 5% bound, so that protocol was not accepted. Prediction fails on a consumer broadband path, under shared-core concurrency, on throttled devices, and when a trace's write rate drifts from its history.

CCS Concepts: • **Information systems** → **Web applications**; • **Networks** → **Network performance modeling**; *Network measurement*; *Network performance analysis*.

Additional Key Words and Phrases: web performance, server-side rendering, client-side rendering, caching, cache invalidation, staleness, performance modeling, prospective validation

## 1 INTRODUCTION

Modern web frameworks let developers choose, per page, whether data is fetched and rendered in the browser (client-side rendering, CSR) or on the server (server-side rendering, SSR), and they offer several caches on each path: a per-user browser cache (e.g. TanStack Query), a shared framework data cache (e.g. the Next.js data cache) and an application cache in front of the database (e.g. Redis). Each layer removes a different part of the request path, and each can show stale data after a write. Which combination is fastest for a page, a network and a server cost, and what it costs in freshness, depends on the request path.

Measurement studies compare rendering strategies under a few network profiles [24] and report stale-read rates of an application cache for a chosen TTL [16]; their numbers describe their

Author's address: Istiak Ahmed Tashrif, istiaktashrif@gmail.com, Department of Computer Science and Engineering, Sonargaon University, Dhaka, Bangladesh.

operating points. What-if predictors such as WebProphet [19] and WebPerf [17] estimate page-load or cloud-side latency under hypothetical changes, and freshness models for caches and replicas [2, 6, 11] predict staleness for one layer at a time. None of them tells a developer, before deploying and from a description of the expected workload, which layer will answer each user action when browser, framework and application caches are stacked, how long the data then takes to appear, and how often it will be out of date.

This paper asks whether a developer can predict the latency–freshness trade-off of alternative rendering and caching policies from a few measurable deployment parameters, and use the prediction to choose a policy. We model each user action (initial load or in-app navigation, CSR or SSR) along its request path: a timeline of the declared sessions determines, per view, which layer serves it (with TTL expiry, stale-while-revalidate, browser revalidation and push invalidation), the data's age when it is shown, and the network and server work on the path. The models are calibrated under network emulation and then used to predict conditions they have not seen, under a prospective protocol: model code and parameters are frozen and hashed, every prediction is written before its run starts, tests use conditions and seeds never used for fitting or diagnosis, criteria are fixed in advance, and an accepted version must pass an independent replication. Every version that failed is reported (Appendix A).

*Contributions.*

(1) A predictor, per user action, of data-visible latency and of the probability that the view shows stale data, for CSR and SSR paths through stacked browser, framework and application caches. It derives which layer serves each view from the declared workload, cache policies and write process (Sections 6–7). The staleness probability is the classical no-change probability of a thinned write process [6, 11]; we apply it to the per-view data age that stacked layers with different semantics produce.

(2) Prospective evidence that this works on conditions never used for fitting: latency within 0.2–1.8% on emulated paths, one real relayed internet path in three configurations (including HTTPS with HTTP/2), a second framework and a heavier page, and, with the serving layer predicted rather than known, under TTL expiry, push invalidation and four write processes (Sections 10 and 11); and on Conduit, an independently written application, on two hosts. Pooled staleness and serving-layer predictions are accurate; per-cell and per-decile criteria were first missed, and after a diagnosed correction the model passed all of them on fresh tests in which it saw only the session distribution. We report every miss with its diagnosis.

(3) An analysis of where the accuracy and the decision quality come from (Sections 10 and 11). Conditioning on the serving layer accounts for most of the latency accuracy; the mechanistic path terms add a smaller margin over an equal-information regression that matters on real paths and needs three or more calibration points. For choosing a policy, a ridge regression given the model's predicted serving layers chooses as well as the model; what simpler predictors lack is the session timeline that predicts serving layers and data age across stacked layers, and without it choices cost more latency or violate the staleness limit (a retrospective comparison).

(4) Policy selection under a staleness limit (Section 8) that chooses, from 27 candidate rendering and caching policies, one within 2% of the observed best in every such test (mean regret 0–0.6%) without violating the staleness limit, as good as per-condition tuning in those tests but without its live pilot runs. With limits placed at the predicted boundary, a margin taken from past errors kept all 16 choices within their limits, at 4.1% and 5.8% mean regret; the

second exceeds our pre-registered 5% bound, a criterion we report as failed. Framework-default TTLs violated the limit in 29 of 32 decisions, and the latency cost of disabling caches ranged from under 3% on Conduit to 21% on the catalogue, depending on the server work a cache saves.

(5) An account of where prediction fails (consumer access links, shared-core concurrency, throttled devices) and of measurement pitfalls that distort such studies (Section 12), and the operating range, costs and deployment steps that follow from the evidence (Section 13).

The individual ingredients are known: what-if latency prediction [17, 19], the no-change probability of a Poisson source [6] and its use for TTL choice [11], and cache-hierarchy analysis [4, 7]. Our contribution is their joint use along the request path of stacked web caches to predict, and choose, before a workload's results are seen, together with prospective evidence of when this works and when it does not.

## 2 BACKGROUND AND RELATED WORK

*CSR and SSR measurements.* Pati and Zaki [24] compare Next.js (SSR) with React (CSR) under three network profiles and report that SSR improves loading on slow networks. Further comparisons report page-load and Core Web Vitals metrics for CSR, SSR and framework rendering modes at single operating points [14, 15, 27, 29, 33], and Chen [5] proposes modular rendering with adaptive hydration. Malavolta et al. [20] measure service-worker caching in progressive web apps. Closest in setting, Gopal [13] evaluates client and Redis caching in a React Native, Next.js and PostgreSQL stack at fixed operating points, without staleness or rendering paths. None of these predicts conditions before measuring them.

*Predicting web performance.* WebProphet [19] extracts object dependencies by perturbing timing and predicts the page-load-time distribution under hypothetical optimizations (errors mostly under 16%). WebPerf [17] predicts the cloud-side latency of a request under what-if changes to resource tiers, including Redis cache tiers, with median errors under 7%, and notes that client and network latency must come from another tool. Ramakrishnan and Kaur [30] and Ghattas et al. [12] predict page performance from page characteristics with statistical and learned models, and Kelton et al. [18] model user-perceived load time. WProf [36] extracts dependency graphs of page loads, which Polaris [22] and Shandian [38] use to speed loads up. These works predict whole-page or server-side latency; in WebPerf a cache is a resource whose speed changes, not a layer whose hit or miss decides the path. We predict, per action, the time until application data is visible end to end, and which layer serves it.

*Freshness and staleness prediction.* Cho and Garcia-Molina [6] model the freshness of a local copy whose source changes as a Poisson process. Quaestor [11] caches query results in web caches with bounded staleness; it treats a result set's records as independent exponential write processes, so the result changes at rate $\sum_i \lambda_{w,i}$, and chooses TTLs for a target probability of seeing a write before expiry, evaluating staleness by simulation. PBS [2] predicts staleness and latency of partial quorums and validates the predictions against a Dynamo-style store. TxCache [26] provides transactional consistency for an application cache; Ji et al. [16] compare TTL and invalidation for an application cache; Mao et al. [21] argue that TTLs are ill-suited to real-time freshness; An et al. [1] study consistency guarantees for caches. Our staleness probability is the same no-change probability as in [6, 11]; we apply it to the data age produced per view by stacked layers with different semantics, predict it before runs, and check it against an oracle that compares every displayed value with the database at display time.

*Cache hierarchies.* Che et al. [4] model hierarchical web caches; Choungmo Fofack et al. [7] and Berger et al. [3] analyze networks of TTL caches; Elsayed et al. [9] optimize TTL hierarchies under network delays. A known consequence is that an upstream cache filters the request stream seen downstream, which we observe in a default stack (Section 5).

*HTTP versions and measurement fidelity.* Wang et al. [37], Varvello et al. [35] and Wijnants et al. [39] show that the benefit of multiplexing and prioritization (HTTP/2 [34]) depends on network and page structure; Rajiullah et al. [28] measure web experience on operational mobile networks; Mahimahi [23] records and replays pages over emulated links; Pediaditakis et al. [25] study the fidelity of reproduced network experiments.

*Search and closest work.* We searched OpenAlex, arXiv, Crossref, IEEE Xplore, the ACM Digital Library and Google Scholar (October 2026; queries, screening criteria and the closest-work comparison with quoted passages are in the artifact). Table 1 compares the closest works by what they predict, from what, for which caches, and how the predictions were checked. What-if latency prediction, Poisson freshness, TTL choice for a staleness target and joint latency–staleness prediction (for replicas) all exist. What none of the works in the table provides together is a prediction, per user action and before the workload runs, of which of several stacked web caches serves it, with the latency and staleness that follow, used to choose a policy and checked prospectively. This is a claim about the literature we searched, not something our experiments show.

## 3 PROBLEM FORMULATION

*Actions and endpoint.* A session is a sequence of actions on one page type: an initial load followed by in-app navigations among pages. The endpoint of an action is its *data-visible time*: from the action's start (navigation start, or a mark immediately before an in-app navigation) until the end of the first frame after the first data row enters the DOM.

*Policies.* A policy $x$ is a rendering mode (CSR or SSR) and a configuration of each cache layer: browser cache (CSR; staleness time $S$), framework data cache (SSR; revalidation time $R$ with stale-while-revalidate, or tag invalidation on write), application cache (Redis; TTL $T$, or deletion on write), each possibly off.

*Staleness.* A view is *stale* if a value it displays differs from the database's value at the instant the view is displayed. The stale-view share of a policy is the fraction of its views that are stale.

*Prediction task.* Before a run, given the frozen calibration, the workload, the policy, the write process, the server work per database read, and quantities measured on the path just before the run (client and internal round-trip times, bottleneck bandwidth, the browser's connection-assignment odds), predict for every action its serving layer (a distribution over browser hit, framework hit, Redis hit, database read), its data-visible time, and its probability of being stale; and, for a set of candidate policies, choose the one with the lowest latency whose predicted stale share is within a limit $\varepsilon$. The workload can be given as the exact sessions (pages per user, in order) or only as their distribution (the access probabilities over pages, the number of users and the session length); in the second case the model averages its predictions over sampled session sets. The write process is given either by its parameters or, for a recorded trace, estimated from a past window. The prediction is therefore conditioned on a workload description, which in a deployment would come from historical traffic.

*Assumptions.* Users are independent browser sessions run one after another (concurrency is treated separately, Section 12); caches start cold; writes update one record each, chosen by the

Table 1. Closest prior work. "Checked against" is how the cited paper evaluated its predictions or measurements; the comparison with quoted passages is in the artifact.

| Work | Predicts | From | Caches | Checked against |
|---|---|---|---|---|
| WebProphet [19] | page-load-time distribution under what-if optimizations | a dependency graph extracted per page by perturbing timing | browser objects; no application caches | measured page loads |
| WebPerf [17] | cloud-side latency of a request under what-if tier changes | profiled dependencies and tier latencies | Redis as a tier whose speed changes, not a hit/miss path | measured cloud-side latency |
| Cho & Garcia-Molina [6] | freshness and age of a local copy | Poisson change rates, refresh policy | one copy | analysis |
| Quaestor [11] | probability of a write before a TTL expires; TTL for a target | per-record write rates of a query result | browser and CDN caches of query results | simulation |
| PBS [2] | staleness and latency of partial quorums | replica latency distributions | none (replicas) | measurements of a Dynamo-style store |
| Pati and Zaki [24], Ji et al. [16] | no prediction; CSR vs SSR, and TTL vs invalidation stale rates | – | not isolated; one application cache | measurement at their operating points |
| This paper | per action: serving layer, data-visible latency, P(stale); a policy under a staleness limit | cache policies, session distribution, write process, pre-run RTT probes, one calibration without writes | browser, framework and application caches, stacked | predictions written before each run; display-time oracle; replications |

declared process; the path is stationary during a run (Section 12 reports what happens when it is not).

## 4 SYSTEM, WORKLOADS AND MEASUREMENT

*Applications.* The first application is a product catalogue on PostgreSQL 17 (5,000 products, 20,000 orders, 50,000 order items, 30,000 reviews, 10,000 users); an Express API serves pages of 25 products with aggregated order counts and ratings, and Redis 7.4 caches responses per page. The front end is Next.js 16.1 (App Router, React 19): the CSR path is a static shell whose data the browser fetches with TanStack Query and navigates with `history.pushState`; the SSR path fetches through the Next.js data cache and returns HTML or a React Server Components payload. A SvelteKit 2 front end implements the same paths, and a heavier variant adds about 300 kB of client JavaScript and four times the payload. The second application is Conduit, the RealWorld reference backend (Express, Prisma 4, PostgreSQL; the public `node-express-realworld-example-app` at a pinned commit), used unmodified: its routes, services, schema and queries answer every request,

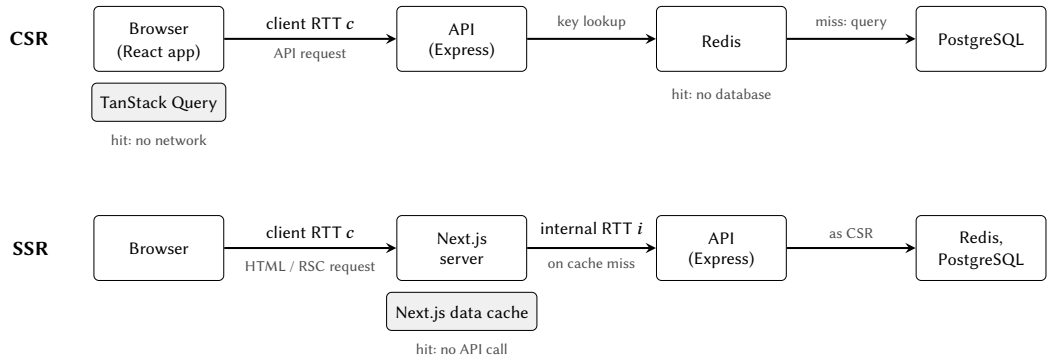



Fig. 1. Request paths of the two rendering modes through the cache layers. Each cache outcome ends the path at a different point; the model's network term counts the client round trip $c$ and internal round trip $i$ on the remaining path. *Initial load*: CSR fetches the document, then the JavaScript bundle (up to six connections per origin; one under HTTP/2), then calls the API on a new connection; SSR fetches the document on a new connection, and the server fetches the data before responding. *Navigation*: CSR changes the URL and makes one API request on a warm connection, or none on a browser-cache hit; SSR makes one RSC request on a warm connection. Data-visible time ends at the first frame after the first data row enters the DOM.

and our entry point only adds request logging, a Redis read-through layer for article reads, a writer and administration endpoints. Its article query joins authors, tags and every favouriting user. The same Next.js front end, with its cache layers and instrumentation, renders article $n$ as one row whose favourites count the oracle checks. Fifty articles are titled after German Wikipedia's 50 most viewed articles of one day, and sessions draw them with probability proportional to those page views; writes favourite articles with the same probabilities, through the application's own service. Popularity and edit timing come from public traces, but the user and write behaviour is synthetic, so these are trace-inspired workloads rather than production ones. Figure 1 shows the request paths.

*Testbed and networks.* All services run in Docker containers; each link (client–web, client–API, web–API) is a separate network whose delay and rate are set with `tc netem`. A Playwright 1.56 Chromium client drives sessions. Real-network tests route the client's traffic through a remote host acting as a layer-3 relay (round trips of about 250–530 ms); one further real path runs from a home broadband connection. Concurrency and CPU-throttling tests pin each service to a dedicated core of a Linux x86 host.

*Measurement.* A `MutationObserver` registered before any body content records the insertion of the first data row, and a `requestAnimationFrame` plus `MessageChannel` callback the next frame. At the insertion instant the page also records every displayed row's identifier and value and the identifier of the API response that produced them.

*Serving layer per view.* Each view is classified by the layer that served it, from identifiers carried out of band: a CSR view without a browser-cache request is a browser hit, otherwise the API log of that request shows a Redis hit or a database read; an SSR view whose displayed response was produced by the API before the view started came from the framework cache, otherwise the API log of that response shows a Redis hit or a database read. (A replay test on the server side alone misclassifies stale-while-revalidate: Section 12.)

*Writes and the staleness oracle.* A seeded writer updates records (catalogue: stock of a product on the workload's pages; every write increments a value) and logs each commit. Push invalidation deletes the Redis key and revalidates the page's framework-cache tag on each write; TTL invalidation relies on expiry. A view is stale if any displayed value differs from the database value at the display instant, reconstructed from a snapshot taken before the writer starts and the write log. Every one of the 52,224 views of the accepted staleness tests captured its full table; no displayed value was ever ahead of the database, also with the write log shifted by ±20 ms, which changes the stale share by at most 0.005.

## 5 PILOT OBSERVATIONS

A pilot (96 runs, 6,912 actions; 2 modes × 4 cache combinations × 2 RTTs × 2 workloads × 3 seeds) motivated the model. Behind the Next.js data cache, Redis's conditional hit ratio was 0.00: a shared upstream cache absorbs every repeated key, the filtering effect of cache-hierarchy theory [4, 7], here in a default configuration; a per-user browser cache leaves Redis almost all of its value. Initial load depends on the rendering path (SSR showed data 153 ms sooner at RTT 0 and 1,093 ms sooner at 170 ms), while caches changed it by under 1%. Navigation has a crossover: with browser caching, CSR navigation beats SSR roughly when the browser hit rate times the client round trip exceeds the server work the shared cache saves.

## 6 LATENCY MODEL

*Structure.* For action $a$ in mode $m$ served by layer $o$, with client round trip $c$ and internal round trip $i$:

$$\hat{T}(m, a, o; c, i) = T_0(m, a, o) + N(m, a, o; c, i) - N(m, a, o; 0, 0) + \rho_{m,a,o}(c), \tag{1}$$

where $T_0$ is the mean latency measured at RTT 0 for that path (CPU, rendering and server time), $N$ is the critical-path network time, and $\rho$ is an empirical residual interpolated over the client RTTs of an emulated calibration grid. An action whose serving layer is uncertain is predicted as $\sum_o P(o)\,\hat{T}(m, a, o)$ (Section 7).

*Critical paths.* CSR initial: document (new connection) → JavaScript bundle → API request (new connection to the API origin). CSR navigation: one API request on a warm connection, or nothing on a browser hit. SSR initial: document (new connection), plus an internal API request on a framework-cache miss. SSR navigation: one RSC request on a warm connection, plus an internal request on a miss.

*Transfers.* A request on a new connection pays a handshake (one RTT for TCP; two with TLS 1.3 [32]). A response of $b$ bytes takes TCP slow-start rounds from an initial window of ten segments [8] (1,448-byte MSS), plus serialization $8b/\text{bw}$.

*The JavaScript bundle and the connection race.* Chromium loads the bundle's chunks over up to six connections per origin; the largest chunk runs either on the reused document connection (whose window has grown) or on a new connection (one more slow-start round), and which becomes free first depends on path micro-timing. The bundle term is $E[\text{bundle}](c) = p\,B_{\text{new}}(c) + (1-p)\,B_{\text{reused}}(c)$, with $B_{\text{new}}, B_{\text{reused}}$ observed per race group in calibration and $p$ measured on the target path before each run from eight probe loads of a page no workload uses. Under HTTP/2 [34] all chunks share one connection and the bundle term is a single calibrated curve.

*Inputs per run.* Frozen calibration; the declared workload; client and internal RTT from TCP-connect probes before the run; the race probability $p$; and, where bandwidth is not fixed, the bottleneck bandwidth (two calibrated bandwidth levels, linear in 1/bw between them).

*Concurrency.* For $N$ concurrent users we extend the model with exact mean value analysis [31] over three single-core stations; queue lengths use each station's total CPU demand per action and each action's latency grows by its on-path demand times the queue length it meets, with load-dependent demand.

## 7 SERVING LAYER AND STALENESS

*Timeline.* The model replays the declared sessions on a timeline whose display gaps (time between consecutive views) are calibrated without writes, per mode, action and serving layer, as a linear function of the client RTT. Each cache entry carries its fill time and $\tau$, the instant at which its data was guaranteed equal to the database. The layers follow their documented semantics: a database read gives $\tau$ = request time; a Redis TTL entry serves while younger than $T$; a push-invalidated entry is deleted on each write to its page; the framework cache with revalidation time $R$ serves its entry and refreshes it in the background once it is older than $R$ (stale-while-revalidate [10]); a browser-cache revisit shows the cached entry and refetches in the background once it is older than $S$. Two timings are approximations: a framework background refresh is taken to replace its entry when it is issued, although it lands only after the internal fetch (which matters only for another request to the same page within those tens of milliseconds), and a request is taken to reach the server half a round trip plus rendering before its data are displayed.

*Serving layer.* Browser and framework TTL hits are determined by the timeline. A push-invalidated entry exists when a view arrives unless a write reached its page since the entry's last access, so the view is served by the next layer down with probability $1 - P_0(\Delta)$, where $\Delta$ is the time since the last access and $P_0$ the write process's no-write probability below. The model thus predicts, per view, a distribution over browser hit, framework hit, Redis hit and database read; with sequential users, cold caches and no expiry within a run, the distribution is degenerate and analytic (criterion C4 of Section 9 checks it).

*Latency with server work.* A view's latency is the calibrated no-write latency of its serving layer plus $d\,P(\mathrm{db})$, where $d$ is the added server work per database read; the display gap grows by the same amount. In the staleness and selection experiments (Section 11), which ran on emulated paths, the no-write latency of each mode, action and serving layer is a line in the client RTT fitted on the calibration runs, not Equation 1; the slow-start and connection-race terms of Section 6 were needed only for the real-path tests of Section 10.

*Staleness semantics.*

Proposition 7.1. *Let the displayed data of a view be equal to the database at instant $\tau$, displayed at instant $t > \tau$. Assume (i) every write changes exactly one record and increases its displayed value, so no value returns to an earlier state; (ii) a write reaches the records shown on the view's page with probability $q$, independently of everything else; (iii) the oracle compares at $t$. Then the view is stale if and only if at least one write reached its page in $(\tau, t]$, and*

$$P(\mathit{stale}) = 1 - P_0(t-\tau), \qquad P_0(w) = E\Big[(1-q)^{N(w)}\Big],$$

*where $N(w)$ is the number of writes of the process in a window of length $w$. For a Poisson process of rate $\lambda$, $P_0(w) = e^{-\lambda q w}$.*

Proof. If no write reaches the page in $(\tau, t]$, every displayed value equals the database at $\tau$ and, unchanged since, at $t$. If at least one does, the record it changed has a larger value at $t$ than at $\tau$ by (i), so the display differs. Conditional on $N(w) = n$, by (ii) none of the $n$ writes reaches the page

with probability $(1-q)^n$; taking the expectation gives $P_0$. For Poisson, $E[s^{N(w)}] = e^{-\lambda w(1-s)}$ with $s = 1-q$. □

The formula is the classical one [6]; Quaestor applies the same thinning to a query's result set [11]. It fails when its assumptions fail: a write that restores an earlier value (an unfavourite after a favourite) can make a page fresh again, so the oracle must compare snapshots rather than count writes; a write to a record that is cached but not displayed does not make the view stale, which (ii) handles only if $q$ counts displayed records; and a page assembled from several responses at different instants has several $\tau$. In our catalogue workload every write increments a stock value of a product on the workload's pages ($q = 1/8$ per page), and all displayed rows come from one response inserted in one DOM mutation, so the proposition applies exactly. In the Conduit workload every write is a favourite by a user who has not favourited that article before; no favourite is removed during a run (favourites are reset to a baseline between runs), so the count only grows and (i) holds with $q$ the article's write probability. Workloads with edits or removals would need the snapshot comparison instead. Even with every cache off, a view can be stale: the data are read at the request and shown about half a round trip plus rendering later; the model predicts this window.

*Other write processes.* For periodic writes every $1/\lambda$ seconds with a uniformly random phase, $N(w) \in \{k, k+1\}$ with $k = \lfloor \lambda w \rfloor$ and $P(N = k+1) = \lambda w - k$. For bursty writes we use a two-state Markov-modulated Poisson process (rate $\lambda_{\text{on}}$ during exponentially distributed ON periods, none during OFF periods), for which $P_0(w) = \pi\, e^{(Q-\Lambda)w}\mathbf{1}$ with generator $Q$, $\Lambda = \operatorname{diag}(q\lambda_{\text{on}}, 0)$ and stationary distribution $\pi$. For a recorded trace we estimate $P_0$ nonparametrically from a training window, as the average of $(1-q)^{N(s,s+w)}$ over start times $s$. These are marginal probabilities; consecutive windows of one session are correlated, which leaves expected stale shares exact but not their variance. We checked each formula against Monte Carlo simulation of its process (agreement within 0.01).

## 8 CHOOSING A POLICY

Given candidate policies $\mathcal{X}$, predicted mean latency $\hat{L}(x)$ and stale share $\hat{S}(x)$, the selector returns

$$x^\star = \arg\min_{x \in \mathcal{X}} \hat{L}(x) \quad \text{subject to } \hat{S}(x) \le \varepsilon,$$

and abstains if no candidate is predicted feasible. A choice is scored on held-out observed values: its *regret* is $(L(x^\star) - L(x^{\text{opt}}))/L(x^{\text{opt}})$, where $x^{\text{opt}}$ minimizes observed latency among observed-feasible candidates, and a choice whose observed stale share exceeds $\varepsilon$ is a *violation*, reported with its magnitude instead of regret. We compare it with a framework-default policy, no caching, the best fixed policy from an earlier experiment, and direct tuning: one pilot run per candidate at the target condition, choosing by pilot means. Tuning needs those runs at every new condition, with live users exposed to every candidate; the model needs one calibration without writes. We do not model monetary cost: CPU work is not money, and our added server work $d$ is a sleep, not CPU consumption.

## 9 METHODOLOGY

*Prospective design.* Each model version is calibrated on emulated data, frozen (code and parameters hashed), and tested on conditions and seeds never used for fitting or diagnosis; the harness writes the prediction for each run before the run starts. A version that passes must pass an independent replication. Failed versions are diagnosed from their data, and the diagnosed data may calibrate the next version, which then faces fresh tests; Appendix A lists every version and every

fresh test. For the experiments added in this revision, three protocols (blocks, seeds, criteria, analysis code) were frozen and committed before their runs: a development block may only check the pipeline, each component has at most three development versions, and the final test and its replication run once. An audit of the record, carried out by the author and therefore not independent, but repeatable from the artifact, checks the commit times against the first run of every block, the frozen manifests against their hashes, the uniqueness of seeds, and the re-scoring of every block from its raw logs. It found two lapses that affected no prediction: an amendment and a change to the calibration script on the second host were committed one and two minutes after the start of the blocks they concern, although both were in place before any of that host's test runs. Appendix B maps every table and figure to its frozen models, blocks, raw logs and regeneration command.

*Latency criteria.* C1: mean relative error over cells $\le 3\%$. C2: every cell within $\max(5\%, 10\,\text{ms})$. C3: every SSR−CSR difference has the correct sign and lies within $\max(10\,\text{ms}, 10\%)$. C4: analytic cache outcomes match 100%. C5: every prediction written before its run started. A cell is a combination of mode, cache setting, action type and network condition; its observed value is the mean over runs.

*Staleness and joint criteria.* W1: pooled mean absolute error over policy cells $\le 0.03$. W2: every cell within 0.05. W3: cells whose predicted stale shares differ by at least 0.05 are ordered correctly. W4: oracle integrity (no displayed value ahead of the database; writes logged; purge lag p99 $\le$ 20 ms). W5: predictions written before runs. L1: mean latency error over (cell, action) $\le 3\%$; L2: every (cell, action) within $\max(5\%, 10\,\text{ms})$.

*Serving-layer criteria.* Predicted serving-layer distributions are scored with the multiclass Brier score $\frac{1}{n}\sum_v \sum_o (p_{v,o} - \mathbf{1}[y_v = o])^2$. O1: in every decile of predicted $P(\text{db})$ holding at least 100 views, the mean prediction is within 0.05 of the observed frequency. O2: the Brier score is below that of calibration-frequency probabilities (class frequencies per policy, action and revisit flag in the calibration runs). O3: the Brier score exceeds the forecaster's own expected score $\frac{1}{n}\sum_v \sum_o p_{v,o}(1 - p_{v,o})$ by at most 0.03. A fixed Brier bound would be unattainable: whether a push-invalidated entry survives is random, and a perfectly calibrated forecaster scores about 0.08 on our workloads.

*Baselines with equal information.* Every baseline is fitted only on the calibration data of the model it is compared with and receives the same inputs per action: mode, action, predicted serving layer, client and internal RTT, bandwidth and CPU level where the model uses them, and the race probability (as an expectation over the two race groups). We fit a ridge regression on serving-layer indicators and their interactions with the inputs (degree 1 or 2), a random forest and gradient-boosted trees, with hyperparameters chosen by leaving one calibration RTT out. Earlier cache-blind baselines (a latency–RTT line per mode and action) are kept for reference.

*Statistics.* The independent unit is the run (one seed of one cell); actions within a run share a session, a cold start and a time window. Intervals are 95% percentile intervals of a block bootstrap that resamples seeds (applied to all cells of a test together, preserving the pairing of methods), 10,000 resamples. With two seeds per cell a bootstrap has almost nothing to resample, so we treat comparisons from such tests (8 of the 16 latency tests: the trusted-certificate HTTP/2, SvelteKit, heavier-page and slow-device tests) as exploratory, whatever their intervals; selection blocks have four or six seeds per cell. Policy decisions are not independent trials: the decisions of one block share one set of observed candidates, so we report the number of distinct candidate sets (scenarios) with every decision count, and worst-case as well as mean regret. The predictors predict means; tail percentiles are descriptive.

Table 2. Every fresh prospective latency test. Mean and max are relative errors over cells; "fails" lists failed criteria; "passed" is the test's own verdict and "accepted" the version's (test and replication both pass).

| Experiment | Conditions | Mean % | Max % | Fails | Passed | Accepted |
|---|---|---|---|---|---|---|
| Emulated (v5) | netem, client 90/210 ms × internal 20/75 ms | 1.10 | 3.78 | – | yes | yes |
| Emulated (v5), replication | netem, client 150/320 × internal 35/100 ms | 0.77 | 3.64 | – | yes | |
| Real path (v7) | relay + 40 ms | 0.61 | 1.15 | – | yes | yes |
| Real path (v7), replication | relay + 20 ms | 1.08 | 2.27 | – | yes | |
| Real path, bandwidth input (v8b) | relay + 10 ms, uncapped | 0.62 | 2.23 | – | yes | yes |
| replication | relay + 40 ms, 15 Mbit/s | 0.70 | 2.38 | – | yes | |
| Real path, HTTPS + HTTP/2 (v9) | relay + 35 ms | 0.26 | 0.44 | – | yes | yes |
| replication | relay + 15 ms | 0.23 | 0.61 | – | yes | |
| HTTP/2, certificate trusted (v9b) | netem 80/330 ms | 1.20 | 6.01 | C2 | no | no |
| replication | netem 200/450 ms | 0.18 | 0.53 | – | yes | |
| SvelteKit | netem 90/300 ms | 0.69 | 2.78 | – | yes | yes |
| replication | netem 200/350 ms | 0.33 | 0.90 | – | yes | |
| Heavier page | netem 90/300 ms | 1.23 | 2.91 | – | yes | yes |
| replication | netem 200/350 ms | 0.61 | 2.27 | – | yes | |
| Slow device, 4× CPU | netem 80/200 ms | 2.28 | 7.05 | C2, C4 | no | no |
| replication, 2× CPU | netem 60/180 ms | 2.08 | 7.22 | C2 | no | |
| Concurrency (P3), N = 4 | netem 100 ms | 3.16 | 3.71 | C1 | no | no |
| replication, N = 8 | netem 100 ms | 2.00 | 3.41 | – | yes | |
| Concurrency (P4), N = 3 | netem 100 ms | 2.26 | 6.35 | C2 | no | no |
| replication, N = 7 | netem 100 ms | 4.74 | 9.37 | C1, C2 | no | |
| Home broadband (H3) | real home path + 5 ms | 8.26 | 26.21 | C1, C2, C3 | no | no |
| replication | real + 25 ms | 5.28 | 11.39 | C1, C2 | no | |

## 10 LATENCY ACCURACY

Table 2 lists every fresh prospective latency test; Fig. 2 shows all of their cells.

*Emulated and real paths.* The accepted emulated model predicts 9,216 actions per test at client RTTs of 90–320 ms and internal RTTs of 20–100 ms with 1.10% (test) and 0.77% (replication) mean error. A model recalibrated on the server host under emulation predicted a real relayed path within

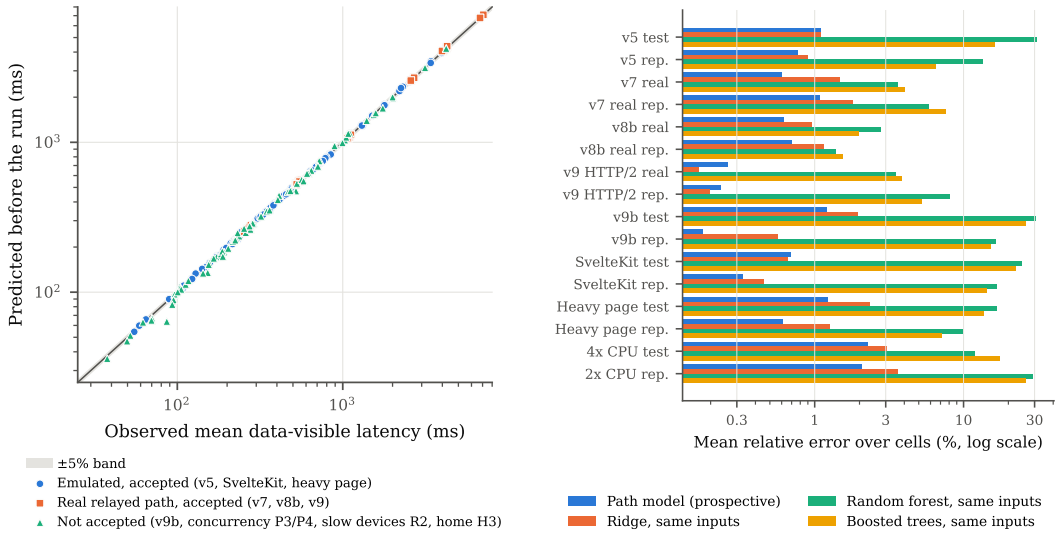



Fig. 2. Left: predicted (written before each run) vs observed mean data-visible latency for every cell of the latency tests in Table 2, grouped by whether the model version was accepted. Right: mean error of the model and of baselines fitted on the same calibration data, per test.

2.3%, but under-predicted CSR initial loads by 6–8%: only the bundle phase differed, because a new connection won the race for the largest chunk in 75% of real loads and 11% of emulated ones (Section 12). With the race probability measured before each run, the model predicts the real path within 0.61% and 1.08%; with bandwidth probed per run, 0.62% and 0.70%; with HTTPS and HTTP/2 through a TLS-terminating edge, 0.26% and 0.23%. Recalibrated, the same equations predict a SvelteKit implementation (0.69%, 0.33%) and a heavier page (1.23%, 0.61%). These numbers hold for emulated paths and for a relayed path between data centres; on a consumer broadband path the errors are 5–8% (Section 12).

*Where the accuracy comes from.* We recomputed every prediction of these tests from the frozen inputs with terms removed (Fig. 3); the full variant reproduces the stored predictions exactly. Without the network and path terms (service time $T_0$ only) the error is 49–96%. Without the fitted residual $\rho$, the HTTP/1.1 tests' error rises from 0.33–1.23% to 0.98–2.81%, so the residual is a small correction there. Replacing the measured race-split bundle curves with an analytic simulation of chunk dispatch raises real-path error to 4.3–4.8%: the connection race must be measured, not simulated. On the original HTTP/2 tests the residual carried 19.3% of accuracy because it absorbed a test-harness certificate artifact (Section 12); with the artifact removed the structure alone reaches 4.6–6.3%. On the home path the residual hurt (5.3–8.3% with it, 3.8–6.4% without). Pooling $T_0$ over serving layers matters only where the server does real work: on the heavier page with 25 ms of database work per read, error rises from 2.2–2.6% to 4.5–7.0%.

*Equal-information baselines.* Cache-blind baselines miss by 10–48%. Once a baseline is told the serving layer, it is competitive: the ridge regression's mean error is 0.17–3.61% across the 16 tests, and the model's is lower in 13 of them. Paired block-bootstrap intervals of the difference exclude zero in favour of the model in 12 tests, include zero in 3, and favour the ridge baseline in 1. Only the 8 tests with four seeds per cell support this comparison: among them the model is better in 5 (all on the real relayed path, and the emulated replication) and tied in 3; the differences in the two-seed

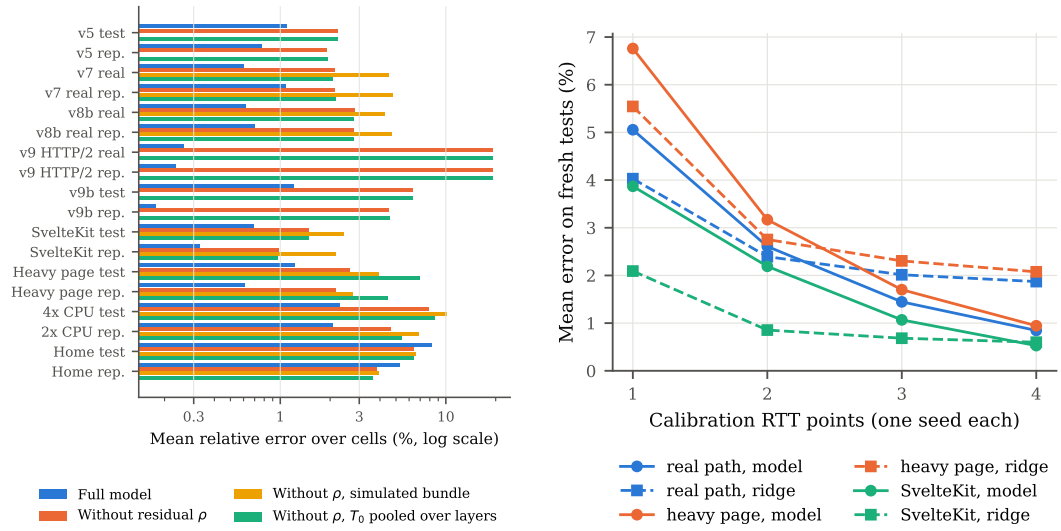



Fig. 3. Left: mean error per test of the full model and with terms removed (retrospective, frozen inputs). Right: mean error on the fresh tests when the model and the equal-information ridge baseline are refitted on fewer calibration RTTs (one seed per point), against calibration runs.

tests are exploratory. The model's advantage is largest on real paths, where initial-load latency is not linear in RTT. Random forests and boosted trees, which cannot interpolate between sparse grid points, miss by 1.4–26.4%. All test conditions lay inside the calibrated RTT range (0 cells outside), so these data cannot show an extrapolation advantage. The conclusion is that conditioning on the serving layer accounts for most of the predictive power; the mechanistic terms add a smaller, real margin.

*Calibration cost.* Refitting on subsets of the calibration grid (Fig. 3, right) shows that one seed per grid point loses nothing, and that the structural model needs three or more RTT points: with three or four points (16–20 runs, under 25 minutes) it beats the ridge baseline on the real path and the heavier page; with one or two points the ridge baseline is better. On SvelteKit the ridge baseline is as good or better at every grid size.

*Tails.* The predictor targets means. On the real paths the 99th percentile of navigation latency is 1.0–1.9 times the predicted mean, so the model should not be read as a tail predictor.

## 11 SERVING LAYERS, STALENESS AND POLICY CHOICE

*Earlier accepted tests.* The staleness model was accepted with the latency model in four fresh tests (Table 3), including a regime test in which each database read costs 25 or 100 ms of server work. Run-level bootstrap intervals of the observed stale share contain the prediction in 31 of 34 cells. Scored retrospectively against the corrected serving-layer classification, the same frozen model predicted which layer served each view with Brier scores of 0.073–0.093 (skill 0.57–0.76 over calibration frequencies; the most likely layer was right for 93–95% of views). On those tests a selector driven by the frozen predictions chose the observed-best feasible policy for every staleness limit (maximum regret 0.0%), but so did two-run tuning: the nine candidate policies differed too much for the choice to be hard.

Table 3. Earlier accepted staleness tests: pooled mean absolute error, worst cell, correct orderings and, for the regime experiment, latency error.

| Test | W1 MAE | W2 max | W3 | L1 | L2 max |
|---|---|---|---|---|---|
| Writes, fast query: test (RTT 150, $\lambda$ 3/s) | 0.0113 | 0.0365 | 24/24 | – | – |
| replication (RTT 60, $\lambda$ 1.5/s) | 0.0090 | 0.0217 | 22/22 | – | – |
| Regime: test (RTT 100, $d$ 25 ms) | 0.0130 | 0.0383 | 24/24 | 1.87% | 4.91% |
| replication (RTT 200, $d$ 100 ms) | 0.0062 | 0.0180 | 27/27 | 0.95% | 2.14% |

Table 4. Prospective tests added in this revision. W1: pooled staleness error; L1: mean latency error; Brier and skill: serving-layer prediction; regret: policy selection (mean over feasible decisions; for the near-threshold blocks, of the selector with a staleness margin; no selector with a margin violated a limit). "Failed" lists the pre-registered criteria the block missed. The near-threshold blocks predict from the session distribution only.

| Block | W1 | L1 % | Brier | Skill | Regret % | Failed |
|---|---|---|---|---|---|---|
| Catalogue, test (RTT 150, $d$ 50 ms) | 0.016 | 1.47 | 0.065 | 0.77 | 0.60 | W2, O1 |
| replication (RTT 80, $d$ 15 ms) | 0.020 | 1.50 | 0.046 | 0.77 | 0.46 | W2, W3, O1 |
| Write processes, test (periodic, on/off, trace) | 0.026 | 1.30 | 0.167 | 0.55 | – | W2, W3, O1 |
| replication | 0.018 | 1.48 | 0.108 | 0.56 | – | W2, W3, O1 |
| Conduit, test (RTT 150) | 0.011 | 1.39 | 0.049 | 0.88 | 0.00 | O1 |
| replication (RTT 80) | 0.013 | 1.39 | 0.034 | 0.91 | 0.17 | W2, W3, O1 |
| Conduit, second host, test (RTT 120) | 0.011 | 0.80 | 0.048 | 0.88 | 0.05 | O1 |
| replication (RTT 60) | 0.012 | 1.34 | 0.034 | 0.91 | 0.55 | W3 |
| Near-threshold selection, model R2, test (RTT 120) | 0.011 | 1.75 | 0.086 | 0.68 | 4.1 | none |
| replication (RTT 200) | 0.013 | 0.96 | 0.075 | 0.75 | 5.8 | B4h-a |

*New prospective tests.* For this revision we froze a protocol and ran new blocks with seeds never used before (Table 4). In the catalogue blocks, 27 candidate policies (three without TTLs; six TTL-based policies at $\tau \in \{1, 5, 20, 60\}$ s, every TTL layer set to $\tau$) run under two access skews (Zipf $\alpha$ 0.6 and 1.0) with Poisson writes and 50 or 15 ms of server work per database read; the test (client RTT 150 ms) and the replication (80 ms) use conditions the model never saw. The model is experiment X's frozen calibration with no refitting; it predicts for every view the serving-layer distribution, the latency and the staleness probability before the run.

*Latency with predicted serving layers.* With the serving layer predicted rather than known, latency remained accurate in every block: 1.47% and 1.50% mean error on the catalogue, 1.30% and 1.48% under other write processes, and 1.39%, 1.39%, 0.80% and 1.34% on Conduit (criteria L1 and L2 passed in all of them). To separate the two sources of error, we re-predicted every view of the 8 selection blocks from the serving layer it was actually served by (retrospectively): criterion L1 is 0.80–1.75% with the predicted layers and 0.77–1.74% with the true ones, at most 0.07 points apart. In these blocks the remaining latency error is error given the layer; mispredicted layers add almost none, because the probabilistic layers (a push-invalidated entry that may or may not have survived) differ in latency mainly by the server work $d$, which the prediction weights by its probability.

*Serving layers.* The predicted layer distributions had Brier scores of 0.065 and 0.046 on the catalogue (skill 0.77 and 0.77 over calibration frequencies; the most likely layer was right for 95.5% and 97.0% of views) and stayed within 0.03 of a calibrated forecaster's own expected score (O3) in

every block. Calibration per probability decile (O1) failed in every block, by up to about 0.07. We diagnosed this after the fact on the catalogue blocks. First, the cache transitions follow the model's rule: for SSR with framework push invalidation, every view that followed a write to its page since the entry's last fill was fetched again (100%), and 97.7% of the others were served from the cache. Second, the write process is right: with the observed time since a page's last access, the Poisson no-write probability gives 0.370 and 0.162 database reads against 0.369 and 0.152 observed, while the model predicted 0.406 and 0.180. About half of that excess came from the model counting an entry's exposure to writes until the view was displayed rather than until the request reached the server (correcting it gives 0.388 and 0.172, so the correction removes about half of the excess; the rest (about 0.02) we have not explained, and candidate causes such as the server's render time before its cache read remain untested. We froze the corrected model (R2) and tested it on two fresh blocks with a new protocol: it passed O1, and every other staleness, latency and serving-layer criterion, in both (Table 4). Third, the criterion was close to the sampling noise: views of one run are correlated, and a bootstrap over runs puts the 95% interval of a decile's deviation at ±0.022–0.054 on the catalogue test, so a perfectly calibrated forecaster would fail a 0.05 bound often; 3 of 9 deciles (test) and 2 of 5 (replication) had intervals excluding zero, with deviations up to 0.066. The serving-layer predictions are therefore calibrated to within about 0.05, with small real biases in a few deciles.

*Staleness.* Pooled staleness errors were 0.016 and 0.020 on the catalogue (W1 passed), but individual cells missed the 0.05 bound (0.052 and 0.082 worst), and two of 1,134 orderings were wrong in the replication; the prediction lay inside the run-bootstrap 95% interval of the observed stale share in 44 of 54 and 45 of 54 cells. Replacing the declared write rate by each run's realized rate changes the pooled error only from 0.016 to 0.015 and from 0.020 to 0.019, so write-rate drift explains little of these misses; the largest ones are in long-TTL cells, where the stale share varies most from run to run. We separate known from estimated write processes. With the process given by its parameters, pooled errors were 0.020 and 0.003 for periodic writes and 0.007 and 0.030 for on/off bursts. With the process estimated from the trace's past window, they were 0.050 and 0.021 (worst cell 0.122): the replayed test segments carried 1.8–2.1 writes per second where the training window implied 2.5, and the rate error accounts for 0.063 of the worst cell's 0.122 miss. A staleness prediction can be no better than its forecast of the write rate; where that forecast is uncertain, the selector below adds a margin rather than trusting the point estimate.

*Workload knowledge.* The predictions above used each run's declared sessions. Re-predicting the same blocks from the access distribution alone (Zipf $\alpha$ or the article popularity, the number of users and the session length; eight sampled session sets per run) left accuracy essentially unchanged: pooled staleness error 0.016, 0.020, 0.010 and 0.013 against 0.016, 0.020, 0.011 and 0.012 with the declared sessions, and run-mean latency error 1.23%, 1.77%, 1.08% and 1.35% against 1.28%, 1.64%, 1.01% and 1.18%. The model needs the distribution of sessions, not the sessions; the near-threshold test below uses distribution-only predictions throughout.

*Choosing a policy.* The selector met its criteria in every block that tested it. On the catalogue, its choices had mean regret 0.60% and 0.46% (worst 2.00%) over 8 feasible decisions per block, with no constraint violation; on Conduit, 0.00% and 0.17%, and on the second host 0.05% and 0.55%. For comparison on the catalogue test: tuning with one pilot run per candidate reached 0.6% (0.5% on the replication) but needs 54 live runs at every new condition; the best fixed policy from the earlier experiment (SSR with framework and Redis push invalidation) 4.4% (2.0%); no caching 21.3% (17.9%); and the framework-default TTL policy violated the staleness limit in 8 of 8 decisions (on Conduit in 3 of 4). On Conduit, whose article reads cost the server little, disabling caches cost only

0.4–2.9%, so the decision there is mostly about freshness; on the catalogue, with 50 or 15 ms of server work per read, it is about both. Selection succeeds although the stale shares it uses are only approximately right, because the candidates' latencies and stale shares are far apart compared with the prediction errors; near a crossover the margin would be smaller.

*Decisions at the threshold.* A selector whose candidates are far apart is easy to get right. To test it where predictions are least certain, a third protocol, frozen before its runs, used 16 candidates chosen to be close (TTLs of 2, 4 and 8 s), a new access skew and server work, distribution-only predictions from model R2, and staleness limits placed at the predicted boundary: for every candidate on the predicted latency–staleness front, one limit 0.01 above and one 0.01 below its predicted stale share, both for all 16 candidates and for the TTL-only subset that an application without write hooks would face (8 decisions per block). The point selector chose the observed-best policy in 5 and 3 of 8 decisions, with mean regret 1.2% and 4.9%, but crossed the limit in 1 and 2 decisions (by at most 0.021; none was confirmed by the run-level interval). A selector that required the predicted stale share plus a margin of 0.05 (the 95th percentile of per-cell staleness error in the earlier blocks) to stay within the limit made no violation in any of the 16 decisions, at mean regret 4.1% and 5.8% (worst 10.5%). The pre-registered bound of 5% on that regret held in the test and was missed in the replication, so the near-threshold protocol as a whole was not accepted. We did not change the bound, the margin or the limits after seeing this result, and we have not run a selector with a re-derived margin: testing one would need a new protocol, frozen from development data, and fresh blocks. Our claim is therefore the narrower one: in 16 near-threshold decisions the margin selector stayed within its limits, at a latency cost that exceeded the pre-registered bound in one of two blocks. The margin is an empirical quantile of earlier errors, not a guarantee: it transfers to a new condition only if errors there resemble the earlier ones, and zero violations in 16 decisions would bound the violation probability below about 17% (one-sided 95%) only if the decisions were independent trials with a common violation probability; they share a model, workloads and conditions, so this is an illustration of how little 16 decisions can establish, not a valid bound. The decisions were also insensitive to its exact value (a post-hoc check): margins of 0.02, 0.03 and 0.05 made the same 16 choices (0 violations, mean regret 4.9%), no margin gave 3 violations at 2.9% regret, and 0.07 abstained once at 5.6%.

*Would a simpler predictor choose as well?* Section 10 showed that a regression told the serving layer predicts latency nearly as well as the model. Whether it also chooses as well is a separate question, which we answered retrospectively on the same blocks (Table 5; this comparison was not pre-registered). Each alternative received only the model's pre-run inputs, was fitted only on the model's calibration runs, and was used with the frozen selection rules, limits and margin; we replaced one component at a time. Given the model's predicted serving layers, a ridge regression on serving-layer indicators and their interactions with the round-trip times chose as the model did: its mean regret differed by at most 0.3 points in any block, and paired run-bootstrap intervals of the difference included zero in all 8 blocks. This is expected, because in these blocks the model's latency per serving layer is itself a calibrated line (Section 7). What the simpler alternatives lack is the session timeline. Without it, serving-layer probabilities must come from the calibration frequencies, which do not change with the TTL; the selector then made no violation (its staleness predictions were still the model's), but its worst regret rose from 2.0% to 13.7% on the 27-candidate blocks and from 11.0% to 15.6% near the threshold. A cache-blind latency predictor cannot tell cache policies of one rendering mode apart; its expected regret was 5.0% (worst 23.4%), with an interval excluding zero in 4 of 8 blocks. Replacing the staleness timeline by the single-layer textbook rule (a cached view's age is uniform on $[0, \mathrm{TTL}]$) kept latency regret low, but violated the limit in 12 of 32 decisions (by up to 0.19) and, even with the 0.05 margin, in 6 of 16 near-threshold

Table 5. Decision quality of the frozen model and of simpler predictors (retrospective, except the model's own choices), with the frozen selection rules. Regret is the mean (worst) over decisions whose choice met the limit; "viol." counts choices whose observed stale share exceeded the limit (a cache-blind predictor's tied choices are scored as a uniformly random pick, so its counts are expectations). 27 candidates: 32 decisions in 8 candidate sets (catalogue and Conduit, both hosts). Near threshold: 16 decisions in 4 candidate sets, selector with the 0.05 margin. Tuning ran one pilot run per candidate (catalogue and local Conduit only, 24 decisions).

| | | | 27 candidates | | Near threshold | |
|---|---|---|---|---|---|---|
| Predictor | Latency | Staleness | Regret % | Viol. | Regret % | Viol. |
| Frozen model | timeline | timeline | 0.4 (2.0) | 0 | 4.9 (11.0) | 0 |
| Ridge, given predicted layers | ridge | timeline | 0.4 (2.0) | 0 | 4.9 (11.0) | 0 |
| Calibration-frequency layers | no timeline | timeline | 1.1 (13.7) | 0 | 6.1 (15.6) | 0 |
| Cache-blind latency | RTT line | timeline | 5.0 (23.4) | 0 | 12.0 (25.7) | 0 |
| Single-layer staleness | timeline | single layer | 0.4 (2.0) | 12 | 2.0 (11.0) | 6 |
| No timeline at all | no timeline | single layer | 0.8 (3.9) | 10 | 2.0 (11.0) | 6 |
| Tuning (pilot runs) | observed at target | | 0.4 (4.2) | 0 | – | – |
| Best fixed earlier policy | – | | 1.8 (13.7) | 0 | – | – |
| Framework-default TTLs | – | | 0.2 (0.4) | 29 | – | – |
| No caching | – | | 10.4 (32.1) | 0 | – | – |

decisions; its violation count exceeded the model's with an interval excluding zero in 5 of 8 blocks. The rule ignores that stale-while-revalidate and browser revalidation show entries older than the TTL, and that the age a view displays depends on its session's revisit pattern. So the model's value for decisions lies in its serving-layer and staleness timeline, not in its latency equations; given that timeline, a regression is an adequate latency component. The alternatives share the model's workload description and calibration, and with 8 and 4 distinct candidate sets these are differences in our setting, not general rankings.

*A second, independently written application.* On Conduit, whose backend, schema, queries and data shape we did not write (its front end is our instrumented Next.js application with the same cache layers, so this tests transfer across backends and workloads, not across front-end frameworks), the model recalibrated on Conduit (36 runs without writes) predicted latency within 1.39% and 1.39% on this machine and 0.80% and 1.34% on a second, Linux host, and its pooled staleness error was 0.011, 0.013, 0.011 and 0.012. Without recalibration, the catalogue's calibration predicted Conduit's latency within 0.76% and 1.87% on the machine it was calibrated on, but missed by 8.06% and 16.47% on the second host, while staleness transferred in both places (pooled error 0.011–0.014): the serving-layer timeline and the write process carry over between applications and hosts, per-layer service times carry over between applications on one host but not between hosts. The second-host blocks each missed one criterion (O1 in the test, W3 in the replication), so cross-host transfer is not accepted under our criteria.

## 12 WHERE PREDICTION FAILS, AND MEASUREMENT PITFALLS

*Consumer broadband.* On a path from a home broadband connection, three model versions missed by 5–8% mean error. Bandwidth varied 3–12 Mbit/s between runs, request RTT drifted by up to 22 ms within a run, and occasional slow requests raised mean navigation latency above any

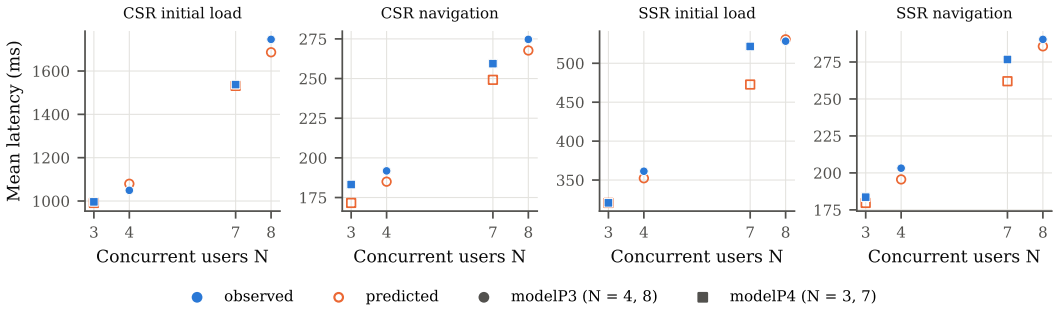



Fig. 4. Concurrency: observed and predicted mean latency per action type for fresh population sizes; two model versions (P3 at $N = 4, 8$; P4 at $N = 3, 7$).

pre-run estimate while typical navigations matched (median 88 ms observed vs 87 ms predicted). We read this as path non-stationarity that pre-run probes cannot forecast; it is a hypothesis consistent with these diagnostics, not a demonstrated cause.

*Concurrency.* With $N$ users sharing one client core and single-core server stations, the queueing extension predicts fresh population sizes ($N = 3, 4, 7, 8$) within 2–5% mean error (Fig. 4); one version passed all criteria at $N = 8$, but no version passed both its test and replication. Navigations are slightly under-predicted throughout; refitting moved errors between population sizes instead of reducing them.

*Throttled devices.* With Chromium CPU throttling calibrated at 1× and 6×, fresh tests at 4× and 2× gave 2.28% and 2.08% mean error, all navigations within 2.3%, but initial loads at 60–80 ms RTT were over-predicted by 4–7%; these RTTs fall between grid points (40 and 120 ms) where initial-load time is not linear in RTT. The HTTP/2 variant with a trusted certificate missed in the same region.

*Measurement pitfalls. Emulation shifts the connection race.* Under `netem` a new connection won the race for the largest chunk in 11% of loads, on the real path in 75%; emulation-based studies of bundle loading should measure connection assignment. *Certificate handling adds two round trips.* Accepting a self-signed certificate through Playwright's `ignoreHTTPSErrors` delays every new TLS connection by about two RTTs (417 ms vs 0.8 ms at RTT 207 ms) and made CSR initial loads 29% slower; trusting the certificate by key removes it. *Stale-while-revalidate defeats replay-based hit detection.* A server-side test that calls a response a cache hit only if it was rendered before misclassifies a background-refreshed entry, which is new but served from cache, as a miss; at 100 ms of server work such "misses" were exactly as fast as hits. Classifying by the displayed response's origin fixes it. *CPU quotas and Apple Silicon distort service times.* CFS quotas throttle bursts (database p99 58 ms vs 17 ms); on an Apple Silicon host the same code ran about 4.5 ms slower per read when lightly loaded, which invalidated a concurrency and a throttling experiment there.

## 13 WHEN SHOULD A DEVELOPER USE THIS PREDICTOR?

*Operating range.* Table 6 summarizes, from the results above, where the predictor was shown to work and where it was not. Every positive entry holds after calibration in that environment; none establishes transfer across hosts or frameworks without recalibration.

Table 6. Validated operating range. "Accepted" means a frozen version passed every pre-registered criterion in a fresh test and its replication.

| Condition | Evidence | Status |
|---|---|---|
| Emulated, stationary paths | latency 0.77–1.10%; staleness and selection blocks on the catalogue (27 candidates) | accepted (latency, selection); per-cell staleness first missed, then passed by R2 |
| One real relayed path (three configurations) | 0.23–1.08% once the connection race and bandwidth are probed before each run; 2.3% and 6–8% CSR initial-load misses without the race probe | accepted; one relay host |
| SvelteKit; heavier page | 0.33–1.23% after recalibration | accepted; two seeds per cell |
| Conduit, calibration host | latency 1.39%, 1.39%; selection regret $\leq$ 0.17%; O1 (and W2, W3 in the replication) missed | latency and selection pass; staleness and calibration criteria not accepted |
| Conduit, second host | latency 0.80%, 1.34% after recalibration; 8.06–16.47% without; one criterion missed per block | not accepted; recalibrate per host |
| Near-threshold selection (R2) | all prediction criteria pass; margin selector 0/16 violations; replication regret 5.8% | prediction accepted; selection protocol not accepted |
| Write-rate drift (trace) | staleness error follows the error of the rate forecast | fails when the rate drifts |
| Consumer broadband | 5–8% mean error | fails |
| Shared-core concurrency | 2–5%; no version passed test and replication | fails |
| CPU-throttled browser | initial loads over-predicted by 4–7% between grid points | fails |

*A deployment workflow.* The experiments suggest the following procedure. It is a recommendation drawn from them, not a production system we evaluated.

(1) Describe the workload from history: the page-access distribution, the number of users and session length, and the write process (a rate, or a trace window). The model needs the distribution of sessions, not the sessions (Section 11).
(2) Calibrate once per application, front end and host, without writes: in our experiments 32 runs (about 33 minutes) for the catalogue's selection model and 36 runs (38–48 minutes) for Conduit; for latency-only predictions on real paths, three or four RTT grid points with one seed each (16–20 runs).
(3) Before predicting, probe the path: TCP-connect round trips and, on real paths, eight probe loads for the connection race and a bandwidth estimate.
(4) Predict serving layers, latency and stale share for every candidate (a fraction of a second per candidate on a laptop) and choose the fastest candidate predicted to meet the limit with a margin taken from past errors; the margin we used cost 4–6% latency near the threshold.
(5) Recalibrate, or abstain, when a condition leaves the validated range: a new host (latency error 8–16% without recalibration on our second host), a write rate unlike its history, an access link whose round trips drift within minutes, or concurrent load on shared cores.

*When it is not worth it.* If only latency matters and the serving layer of each action is known, a regression on the serving layer is as accurate, and with one or two calibration points more accurate (Section 10). If live tuning is acceptable, one pilot run per candidate matched the model in our tests (mean regret 0.4%), at 27–54 runs at every new condition with users exposed to every candidate; the predictor pays off when conditions change more often than a team can afford to tune, or when exposing users to stale candidates is unacceptable. When a cache saves the server little work, as on Conduit, where disabling caches cost under 3%, the choice is mostly about freshness, and push invalidation, where the application can hook its writes, removes most staleness without a model; framework-default TTLs, in contrast, violated the limit in 29 of 32 decisions.

*What the endpoint measures.* Data-visible time ends at the first frame after the first application data row enters the DOM. It is not Largest Contentful Paint, full page load or interaction latency: it excludes images, layout of the rest of the page and hydration after the data appears, and it is measured in headless Chromium. We make no claim about those metrics or about perceived responsiveness.

## 14 THREATS TO VALIDITY

Two applications, one of them on an independently written backend but sharing our instrumented Next.js front end, and their variants (a second front-end framework, SvelteKit, was tested on the catalogue only); headless Chromium only; real paths through one relay host and one home connection; the trace-driven workloads combine public timing or popularity data with synthetic record choice, so they are trace-inspired rather than production workloads; models are recalibrated per framework, application and environment; the heavier-page tests ran with 25 ms of added database work per read in calibration and test. Predictions are conditioned on a workload description (the session distribution and the write process); when the write rate drifts from its history, staleness predictions drift with it. Prediction intervals and cache eviction under memory pressure were not evaluated. The comparison of the model's decisions with simpler predictors, the true-layer latency analysis, the ablation and the equal-information baselines are retrospective analyses of prospectively collected data; only the model's own predictions and choices were pre-registered. The audit of the record was carried out by the author, not independently.

## 15 CONCLUSION

*What was shown.* After a one-time calibration per environment and short network probes, data-visible latency and staleness of cached CSR and SSR applications were predicted before each run from the request path: which layer answers, the round-trip times, the bytes, the bandwidth, the server's work and the write process. Knowing which layer answers is most of the latency prediction, and it can itself be predicted from a description of the workload and the cache policies. Used to choose among 27 rendering and caching policies under a staleness limit, on two applications and two hosts, the predictions chose about as well as live tuning did without running the candidates, and made no limit violation. The session timeline, not the latency equations, carries this decision quality: given its serving layers, a regression chooses as well; without it, simpler predictors choose slower policies or violate the limit.

*What was not shown.* The near-threshold selector met its staleness limits but missed its pre-registered regret bound in one of two blocks. The predictions first missed some per-cell and calibration criteria; a corrected model passed them on two fresh tests, which is evidence for that model on those conditions, not a general guarantee. Prediction failed on a consumer broadband path, under shared-core concurrency, on throttled devices and when the write rate drifted from its

history, and transfer across hosts required recalibration. The comparisons with simpler predictors are retrospective.

*When it is useful.* The predictor is worth its calibration when a team must choose among many rendering and caching policies under a freshness requirement, on stationary paths and with a workload whose sessions and writes resemble their history, and cannot afford to tune every candidate live at every new condition. Because every prediction was written before its run, each of these conclusions, including the negative ones, was checked on conditions the models had not seen.

## ARTIFACT AVAILABILITY

The testbed, harness, frozen models and protocols, per-run predictions, raw logs and evaluation outputs, the analysis scripts and a script that re-scores every test from the raw logs are archived on Zenodo (doi:10.5281/zenodo.23512120, which resolves to the latest version; this paper uses version 1.2.0) and developed at https://github.com/Istiak-A-Tashrif/cached-csr-ssr-latency.

## ACKNOWLEDGMENTS

The author used a generative AI tool (Claude, Anthropic) to assist with implementing the testbed and analysis code, running and analyzing the experiments, producing the figures, and drafting and editing the text. The author directed the research and is responsible for all content.

## REFERENCES

[1] S. An, Y. Cao, J. Li, and Y. Li. 2026. Caching with Monotonicity and Consistency. *ACM Transactions on Database Systems* (2026). https://doi.org/10.1145/3845609

[2] Peter Bailis, Shivaram Venkataraman, Michael J. Franklin, Joseph M. Hellerstein, and Ion Stoica. 2012. Probabilistically Bounded Staleness for Practical Partial Quorums. *Proceedings of the VLDB Endowment* 5, 8 (2012), 776–787. https://doi.org/10.14778/2212351.2212359

[3] Daniel S. Berger, Philipp Gland, Sahil Singla, and Florin Ciucu. 2014. Exact Analysis of TTL Cache Networks. *Performance Evaluation* (2014). https://doi.org/10.1016/j.peva.2014.07.001

[4] Hao Che, Ye Tung, and Zhijun Wang. 2002. Hierarchical Web Caching Systems: Modeling, Design and Experimental Results. *IEEE Journal on Selected Areas in Communications* 20, 7 (2002). https://doi.org/10.1109/JSAC.2002.801752

[5] Kaitao Chen. 2025. Improving Front-end Performance through Modular Rendering and Adaptive Hydration (MRAH) in React Applications. arXiv:2504.03884 [cs.SE]

[6] Junghoo Cho and Hector Garcia-Molina. 2000. Synchronizing a Database to Improve Freshness. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. 117–128. https://doi.org/10.1145/342009.335391

[7] Nicaise Choungmo Fofack, Philippe Nain, Giovanni Neglia, and Don Towsley. 2014. Performance Evaluation of Hierarchical TTL-based Cache Networks. *Computer Networks* (2014). https://doi.org/10.1016/j.comnet.2014.03.006

[8] J. Chu, N. Dukkipati, Y. Cheng, and M. Mathis. 2013. Increasing TCP's Initial Window. RFC 6928. https://doi.org/10.17487/RFC6928

[9] Karim S. Elsayed, Fabien Geyer, and Amr Rizk. 2024. Utility-Driven Optimization of TTL Cache Hierarchies Under Network Delays. In *2024 IFIP Networking Conference (IFIP Networking)*. https://doi.org/10.23919/IFIPNetworking62109.2024.10619843

[10] R. Fielding, M. Nottingham, and J. Reschke. 2022. HTTP Caching. RFC 9111. https://doi.org/10.17487/RFC9111

[11] Felix Gessert, Michael Schaarschmidt, Wolfram Wingerath, Erik Witt, Eiko Yoneki, and Norbert Ritter. 2017. Quaestor: Query Web Caching for Database-as-a-Service Providers. *Proceedings of the VLDB Endowment* 10, 12 (2017). https://doi.org/10.14778/3137765.3137773

[12] Mohammad Ghattas, Antonio M. Mora, and Suhail Odeh. 2025. A Novel Approach for Evaluating Web Page Performance Based on Machine Learning Algorithms and Optimization Algorithms. *AI* 6, 2 (2025), 19. https://doi.org/10.3390/ai6020019

[13] A. Gopal. 2026. Client and Server Caching Strategies in Modern Mobile-First Architectures: React Native, Next.js, and PostgreSQL. SSRN preprint. https://doi.org/10.2139/ssrn.6602728

[14] Roy Hanafi, Abd Haq, and Ninik Agustin. 2024. Comparison of Web Page Rendering Methods Based on Next.js Framework Using Page Loading Time Test. *Teknika* 13 (2024), 102–108. https://doi.org/10.34148/teknika.v13i1.769

[15] Taufan Fadhilah Iskandar, Muharman Lubis, Tien Fabrianti Kusumasari, and Arif Ridho Lubis. 2020. Comparison between Client-Side and Server-Side Rendering in the Web Development. *IOP Conference Series: Materials Science*

*and Engineering* 801 (2020), 012136. https://doi.org/10.1088/1757-899X/801/1/012136

[16] Yunhong Ji, Xuan Zhou, Yongluan Zhou, and Ke Wang. 2023. The Case of Transparent Cache Invalidation in Web Applications. arXiv:2311.02384 [cs.DB]

[17] Yurong Jiang, Lenin Ravindranath Sivalingam, Suman Nath, and Ramesh Govindan. 2016. WebPerf: Evaluating What-If Scenarios for Cloud-hosted Web Applications. In *Proceedings of the 2016 ACM SIGCOMM Conference*. 258–271. https://doi.org/10.1145/2934872.2934882

[18] Conor Kelton, Jihoon Ryoo, Aruna Balasubramanian, Xiaojun Bi, and Samir R. Das. 2020. Modeling User-Centered Page Load Time for Smartphones. In *22nd International Conference on Human-Computer Interaction with Mobile Devices and Services (MobileHCI '20)*. ACM. https://doi.org/10.1145/3379503.3403565

[19] Zhichun Li, Ming Zhang, Zhaosheng Zhu, Yan Chen, Albert Greenberg, and Yi-Min Wang. 2010. WebProphet: Automating Performance Prediction for Web Services. In *7th USENIX Symposium on Networked Systems Design and Implementation (NSDI '10)*. USENIX Association, San Jose, CA.

[20] Ivano Malavolta, Katerina Chinnappan, Lukas Jasmontas, Sarthak Gupta, and Kaveh Ali Karam Soltany. 2020. Evaluating the Impact of Caching on the Energy Consumption and Performance of Progressive Web Apps. In *Proceedings of the IEEE/ACM 7th International Conference on Mobile Software Engineering and Systems (MOBILESoft '20)*. https://doi.org/10.1145/3387905.3388593

[21] Ziming Mao, Rishabh Iyer, Scott Shenker, and Ion Stoica. 2024. Revisiting Cache Freshness for Emerging Real-Time Applications. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks (HotNets '24)*. ACM. https://doi.org/10.1145/3696348.3696858

[22] Ravi Netravali, Ameesh Goyal, James Mickens, and Hari Balakrishnan. 2016. Polaris: Faster Page Loads Using Fine-grained Dependency Tracking. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI '16)*. USENIX Association.

[23] Ravi Netravali, Anirudh Sivaraman, Somak Das, Ameesh Goyal, Keith Winstein, James Mickens, and Hari Balakrishnan. 2015. Mahimahi: Accurate Record-and-Replay for HTTP. In *2015 USENIX Annual Technical Conference (USENIX ATC '15)*. USENIX Association.

[24] Swostik Pati and Yasir Zaki. 2025. Evaluating the Efficacy of Next.js: A Comparative Analysis with React.js on Performance, SEO, and Global Network Equity. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*. ACM. https://doi.org/10.1145/3701716.3715565

[25] Dimosthenis Pediaditakis, Charalampos Rotsos, and Andrew W. Moore. 2014. Faithful Reproduction of Network Experiments. In *Proceedings of the Tenth ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS '14)*. 41–52. https://doi.org/10.1145/2658260.2658274

[26] Dan R. K. Ports, Austin T. Clements, Irene Zhang, Samuel Madden, and Barbara Liskov. 2010. Transactional Consistency and Automatic Management in an Application Data Cache. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10)*. USENIX Association.

[27] N. P. Putri Cahyono, H. Hoiriyah, A. K. Darmawan, and M. Rofiuddin. 2026. Performance Analysis of Client-Side Rendering in a Hybrid LMS using Core Web Vitals Under Diverse Network Conditions. *SISTEMASI* 15 (2026). https://doi.org/10.32520/stmsi.v15i7.6500

[28] Mohammad Rajiullah, Andra Lutu, Ali Safari Khatouni, Mah-Rukh Fida, Marco Mellia, Anna Brunstrom, Ozgu Alay, Stefan Alfredsson, and Vincenzo Mancuso. 2019. Web Experience in Mobile Networks: Lessons from Two Million Page Visits. In *The World Wide Web Conference (WWW '19)*. 1532–1543. https://doi.org/10.1145/3308558.3313606

[29] M. R. Ramadhan, S. Suakanto, E. N. Alam, and E. T. Nuryatno. 2026. Performance Comparison of Next.js (SSR) and React (CSR) under Dynamic Loading Strategies. In *2026 13th International Conference on ICT for Smart Society (ICISS)*. IEEE. https://doi.org/10.1109/ICISS71466.2026.11715518

[30] Raghu Ramakrishnan and Arvinder Kaur. 2020. An Empirical Comparison of Predictive Models for Web Page Performance. *Information and Software Technology* 123 (2020), 106307. https://doi.org/10.1016/j.infsof.2020.106307

[31] M. Reiser and S. S. Lavenberg. 1980. Mean-Value Analysis of Closed Multichain Queuing Networks. *J. ACM* 27, 2 (1980). https://doi.org/10.1145/322186.322195

[32] E. Rescorla. 2018. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. https://doi.org/10.17487/RFC8446

[33] Mykola Savenko and Kateryna Babii. 2025. Performance Optimization Strategies for Large-Scale Web Applications Using Next.js. *IEEE Access* 13 (2025), 217376–217386. https://doi.org/10.1109/ACCESS.2025.3647563

[34] M. Thomson and C. Benfield. 2022. HTTP/2. RFC 9113. https://doi.org/10.17487/RFC9113

[35] Matteo Varvello, Kyle Schomp, David Naylor, Jeremy Blackburn, Alessandro Finamore, and Konstantina Papagiannaki. 2016. Is the Web HTTP/2 Yet?. In *Passive and Active Measurement (PAM 2016) (Lecture Notes in Computer Science)*. Springer. https://doi.org/10.1007/978-3-319-30505-9_17

[36] Xiao Sophia Wang, Aruna Balasubramanian, Arvind Krishnamurthy, and David Wetherall. 2013. Demystifying Page Load Performance with WProf. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI*

*'13)*. USENIX Association.
[37] Xiao Sophia Wang, Aruna Balasubramanian, Arvind Krishnamurthy, and David Wetherall. 2014. How Speedy is SPDY?. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI '14)*. USENIX Association.
[38] Xiao Sophia Wang, Arvind Krishnamurthy, and David Wetherall. 2016. Speeding up Web Page Loads with Shandian. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI '16)*. USENIX Association.
[39] Maarten Wijnants, Robin Marx, Peter Quax, and Wim Lamotte. 2018. HTTP/2 Prioritization and its Impact on Web Performance. In *Proceedings of the 2018 World Wide Web Conference (WWW '18)*. 1755–1764. https://doi.org/10.1145/3178876.3186181

## A EVERY MODEL VERSION AND FRESH TEST

Table 7 lists every model version, the reason it was created, the fresh tests it faced and the outcome. Diagnosed data from a failed test may calibrate the next version but never scores it.

Table 7. Model versions and their fresh tests (mean / max relative error, %, and failed criteria).

| Version | Change and motivation | Fresh tests (mean / max %) | Outcome |
|---|---|---|---|
| *Latency, emulated paths (macOS host)* | | | |
| v1 | Pilot path model | 1 test (max 39.4) | Superseded before criteria existed |
| v2 | Analytic cache outcomes; slow-start transfers | 1 test (3.3 / 11.8) | Superseded before criteria existed |
| v3 | TCP-connect RTT probes; warm-up page | stopped part-way | Stopped (author decision) |
| v4 | Connection-aware bundle dispatch; serialization | 3.68 / 11.88 (C1, C2) | Not accepted; criteria fixed before scoring |
| v5 | Residual $\rho$ over client RTT | 1.10 / 3.78; rep 0.77 / 3.64 | **Accepted** |
| *Latency, real relayed path (Linux host)* | | | |
| v6 | v5 recalibrated under emulation on the server host | 2.28 / 7.74 (C2, C3); rep 2.34 / 7.01 (C2) | Not accepted: connection race |
| v7 | Race probability measured before each run | 0.61 / 1.15; rep 1.08 / 2.27 | **Accepted** |
| v8 | Bandwidth as a measured input | 0.64 / 2.04; rep 0.88 / 2.51 (C3) | Not accepted: SSR navigation offset |
| v8b | v8 plus an SSR-navigation transfer offset from used real-path runs | 0.62 / 2.23; rep 0.70 / 2.38 | **Accepted** |
| v9 | HTTPS + HTTP/2 (TLS handshake, one connection) | 0.26 / 0.44; rep 0.23 / 0.61 | **Accepted** (residual absorbed a certificate artifact) |
| v9b | v9 with the certificate trusted | real path invalid (RTT 490–1,700 ms); emulated 1.20 / 6.01 (C2); rep 0.18 / 0.53 | Not accepted (miss below the grid) |
| *Latency, transfer and limits* | | | |
| S | v7 equations recalibrated for SvelteKit | 0.69 / 2.78; rep 0.33 / 0.90 | **Accepted** |
| LQ | v7 equations for a heavier page | 1.23 / 2.91; rep 0.61 / 2.27 | **Accepted** |
| R | CPU throttling, two calibrated levels (macOS) | 3.26 / 5.05; rep 6.18 / 12.44 | Invalid: load-dependent core speed |
| LR2 | R repeated on pinned Linux cores | 2.28 / 7.05 (C2, C4); rep 2.08 / 7.22 (C2) | Not accepted; iteration stopped |
| C | Concurrency, MVA (macOS) | 10.79 / 13.61; rep 15.92 / 18.68 | Invalid: load-dependent core speed |
| P | MVA on pinned cores | 11.98 / 38.17; rep 26.56 / 83.78 | Not accepted: launch bursts, total demand |
| P2 | Launch barrier; on-path vs total demand | 2.70 / 8.84 (C2); rep 8.86 / 15.95 | Not accepted: load-dependent demand |
| P3 | Load-dependent demand | 3.16 / 3.71 (C1); rep 2.00 / 3.41 | Not accepted (replication passed) |

| Version | Change and motivation | Fresh tests (mean / max %) | Outcome |
|---|---|---|---|
| P4 | P3 refitted on all barrier-era runs | 2.26 / 6.35 (C2); rep 4.74 / 9.37 | Not accepted; iteration stopped |
| H | Home broadband path | 6.76 / 7.90; rep 3.69 / 8.63 | Not accepted |
| H2 | Bandwidth levels; request-level RTT | 5.31 / 11.54; rep 6.36 / 22.58 | Not accepted: short probe read low |
| H3 | Pinned server cores; bundle-equivalent bandwidth | 8.26 / 26.21; rep 5.28 / 11.39 | Not accepted; iteration stopped |
| *Staleness (pooled absolute error)* | | | |
| W | Timeline staleness model | 0.063 (application bug); rep discarded unscored | Bug fixed; model unchanged |
| W (re-test) | Same frozen model, 8 seeds per cell | 0.011; rep 0.009 | **Accepted** |
| X | Joint latency and staleness with server work $d$ | 0.013, L1 1.87; rep 0.006, L1 0.95 | **Accepted** |
| *Revision: serving layer, staleness and selection (protocol frozen before the runs)* | | | |
| R | Experiment X's frozen calibration; declared write process; serving-layer distribution per view | catalogue 1.47/ 1.50 (latency); write processes 1.30/ 1.48 | Selection accepted; staleness and serving-layer gates not accepted (W2, O1; W2, W3, O1; W2, W3, O1; W2, W3, O1) |
| RC | R with per-page write probability; recalibrated on Conduit (36 runs) | Conduit 1.39/ 1.39; second host 0.80/ 1.34 | Selection accepted; other gates not accepted (O1; W2, W3, O1; O1; W3) |
| R2 | R with the push-exposure timing correction found post hoc on the catalogue blocks; predictions from the session distribution only | near-threshold 1.75/ 0.96 (latency) | All prediction criteria passed in both; margin selector 0 violations; regret bound missed in the replication |

## B REPRODUCIBILITY MANIFEST

Table 8 maps every table and figure to its frozen models, experiment blocks, raw logs and the command that regenerates it. Paths are relative to `testbed/` in the artifact. The script `tools/reproduce_evaluations.sh` re-scores every block below from its raw logs, reruns the decision comparison, compares each output with the stored one and checks the SHA-256 sums of the raw data; the frozen models' hashes and the protocols' commit times are in `protocol/FROZEN*.json` and `protocol/AUDIT.md`. Analysis outputs feed the text through `paper/make_numbers.py` and the figures and generated tables through `paper/make_figs.py`; no reported number is typed by hand, except in the version ledger (Table 7), which is maintained by hand and checked against the audit.

Table 8. Reproducibility manifest.

| Item | Frozen models | Blocks | Raw logs | Regenerated by |
|---|---|---|---|---|
| Table 2, Fig. 2 (left) | 5, 7, 8b, 9, 9b, S, LQ, LR2, P3, P4, H3 | prosp5/rep5, real7/rep7, real8b/rep8b, real9/rep9, test9b/reptest9b, testS/repS, testQ/repQ, testR2/repR2, concP3/repP3, concP4/repP4, realH3/repH3 | `results/`, `server-final/`, `server-R2/` | `evaluate_n.py`; `make_figs.py` |
| Table 3 | W, X | testW2/repW2, testX/repX | `results/` | `evaluateW.py`, `evaluateX.py` |

| Item | Frozen models | Blocks | Raw logs | Regenerated by |
|---|---|---|---|---|
| Table 4, Section 11 | R, RC (calibration host and second host), R2 | revT/revR, revB2T/revB2R, cdT/cdR, cdHT/cdHR, revHT/revHR; pilots revPilotT/R, cdPilotT/R | `results/`, `server-cd/` | `evaluateR.py`, `evaluateRC.py`, `evaluateR2.py` |
| Table 5; true-layer latency | R, RC, R2 (calibrations calibW, cdCal, cdCalH) | the eight selection blocks | as above | `decision_baselines.py` |
| Fig. 2 (right); equal-information baselines | as Table 2 | as Table 2, without the home path | as Table 2 | `baselines_eq.py`, `uncertainty.py` |
| Fig. 3 | as Table 2 | HTTP/1.1, HTTP/2, SvelteKit, heavier-page and home tests | as Table 2 | `ablation.py`, `calib_efficiency.py` |
| Fig. 4 | P3, P4 | concP3/repP3, concP4/repP4 | `server-final/` | `evaluate_n.py`; `make_figs.py` |
| Oracle and serving-layer diagnoses | R, R2 | revT/revR, revB2T/revB2R, revHT/revHR | `results/` | `staleness_checks.py`, `diag_push.py`, `diag_o1_noise.py`, `diag_stale.py`, `diag_decomp.py`, `diag_distonly.py`, `diag_margin.py` |